AI PORTFOLIO PROJECT SUMMARY

1. Project Overview

This project is a production-ready AI RAG (Retrieval-Augmented Generation) Chat Widget integrated into a personal portfolio website. It acts as an intelligent virtual agent that can answer recruiter or user queries in real-time based on the owner's CV and technical blog posts, leveraging modern AI infrastructures.

2. System Architecture

The system is built as a separated Client-Server model (Monorepo architecture structure):

- **Front-End (Client):** Developed with React/Next.js and styled with Tailwind CSS. It handles the interactive user interface, message state management, and real-time streaming data parsing. Deployed globally on Vercel.
- **Back-End (Server):** Powered by Python FastAPI. It coordinates the RAG logic, manages document data ingestion, and exposes secure API endpoints. Deployed on Render.
- **Vector Store & AI Model:** Utilizes Google Gemini API (text-embedding-004) for embedding generation and (gemini-1.5-flash) for context-aware response synthesis. Local storage is handled via FAISS (with scalability routes designed for Pinecone Cloud).

3. Core Technical Concepts

- **RAG (Retrieval-Augmented Generation):** Dynamically injects relevant document snippets directly into the LLM context loop to prevent hallucinations and provide accurate, source-grounded answers.
- **Data Flow Pipeline:** User Query → API Request → Query Embedding → FAISS Similarity Search → Prompt Synthesis → Gemini Response → Front-End Stream Output.
- **Production Security:** CORS configuration restricts untrusted external requests while exposing key endpoints securely via environment variables (.env masking).

4. Production Deployment Checklist

Component	Platform	Build Command	Start Command / Root
Front-End	Vercel	npm run build	Root: frontend/
Back-End	Render	pip install -r requirements.txt	uvicorn main:app --host 0.0.0.0 --port $PORT Root: backend/

5. Scalability & Advanced Enhancements

- **Pinecone Cloud Upgrade:** Migrating from FAISS to Pinecone to ensure persistent cloud index storage, avoiding cold-start re-indexing overhead from Render ephemeral storage container life-cycles.
- **Redis Layer:** Implementing an Upstash Redis Caching cluster to intercept identical queries, dropping LLM token usage to 0 and reducing response latencies to milliseconds.

- **Markdown & Syntax Highlight:** Integrating react-markdown and prismjs into the React UI layer to provide premium IDE-grade theme formatting for raw code snippets generated by the assistant.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.